KRUTIK 🚨 CYBER EXPERT

CYBER SECURITY BASICS

Ethical Hacking | Defense | Secure Coding

BY KRUTIK 🚨
CYBER EXPERT

"Learn Free. Hack Ethical. Secure World."

CHAPTER 1: CYBER SECURITY INTRODUCTION

What is Cyber Security?
- Practice of protecting systems, networks, and data from digital attacks
- CIA Triad: Confidentiality, Integrity, Availability
- Also known as Information Technology (IT) Security

Why Cyber Security?
✅ 2,200+ cyber attacks happen every day globally
✅ Average cost of a data breach: $4.45 Million (2023)
✅ 95% of breaches are caused by human error
✅ Cyber crime will cost $10.5 Trillion annually by 2025
✅ Every company needs security experts

Types of Hackers:

Type	Description
White Hat	Ethical hackers (good guys)
Black Hat	Malicious hackers (bad guys)

Grey Hat	In-between (may break laws but not malicious)
Script Kiddie	Uses existing tools (noob)
Hacktivist	Hacks for political/social causes
State-Sponsored	Government-backed hackers

CHAPTER 2: COMMON CYBER ATTACKS

A) PHISHING ATTACKS:

CODE:

```
What is Phishing?
- Fake emails/websites that trick users into giving sensitive info
- Most common attack (80% of security incidents)

Types:
1. Email Phishing - Fake emails from banks, companies
2. Spear Phishing - Targeted at specific person
3. Whaling - Targeted at CEOs/Executives
4. Smishing - SMS phishing
5. Vishing - Voice phishing (phone calls)

How to Prevent:
✅ Check sender email address carefully
✅ Don't click on suspicious links
✅ Look for spelling/grammar errors
✅ Verify with the company directly
✅ Use email filtering
```

B) MALWARE (Malicious Software):

CODE:

```
Types of Malware:
1. Virus - Self-replicating, attaches to files
2. Worm - Self-replicating, spreads over networks
3. Trojan - Disguised as legitimate software
```

4. Ransomware - Encrypts files, demands payment
5. Spyware - Steals information secretly
6. Adware - Displays unwanted ads
7. Rootkit - Hides deep in system

Prevention:
✅ Install antivirus/anti-malware
✅ Keep software updated
✅ Don't download from untrusted sources
✅ Use application whitelisting
✅ Regular backups

C) MAN-IN-THE-MIDDLE (MITM) ATTACKS:

CODE:
_____

What is MITM?
- Attacker intercepts communication between two parties
- Can eavesdrop, modify, or steal data

Common MITM Scenarios:
1. Public Wi-Fi (evil twin) attacks
2. DNS spoofing
3. HTTPS downgrade attacks
4. Session hijacking

Prevention:
✅ Use HTTPS (SSL/TLS)
✅ Avoid public Wi-Fi for sensitive transactions
✅ Use VPN
✅ Check SSL certificates
✅ Enable HSTS

D) SQL INJECTION:

CODE:
_____

What is SQL Injection?
- Attacker injects malicious SQL code into input fields
- Can read, modify, or delete database data

Example (Vulnerable Code):
-- Bad: Direct concatenation
$sql = "SELECT * FROM users WHERE username = '" . $username . "'";

-- Attacker inputs: ' OR '1'='1
-- Becomes: SELECT * FROM users WHERE username = '' OR '1'='1'

Example (Secure Code):
-- Good: Prepared statements
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");
$stmt->execute([':username' => $username]);

Prevention:
✅ Use prepared statements (PDO)
✅ Input validation and sanitization
✅ Least privilege principle
✅ Use ORM frameworks

E) CROSS-SITE SCRIPTING (XSS):

CODE:
_____

What is XSS?
- Attacker injects malicious JavaScript into web pages
- Steals cookies, session tokens, or redirects users

Types:
1. Stored XSS - Saved in database (permanent)
2. Reflected XSS - Reflected in URL (temporary)
3. DOM-based XSS - Client-side vulnerability

Example (Vulnerable):
-- Bad: Direct output
echo $_GET['name'];

Example (Secure):
-- Good: Escape output
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');

Prevention:
✅ Always escape output (htmlspecialchars)
✅ Content Security Policy (CSP)
✅ Validate input
✅ Use XSS protection headers

F) CROSS-SITE REQUEST FORGERY (CSRF):

CODE:
_____

What is CSRF?
- Attacker tricks user into performing unwanted actions
- Uses user's authenticated session

Example:
- User is logged into bank.com
- Attacker sends a malicious link
- User clicks link → money is transferred without consent

Prevention:
✅ CSRF tokens
✅ SameSite cookies
✅ Re-authentication for sensitive actions
✅ Double-submit cookies

G) DENIAL OF SERVICE (DoS/DDoS):

CODE:

What is DoS?
- Overwhelming a service to make it unavailable
- DDoS: Distributed (multiple sources)

Types:
1. Volume-based - Flood with traffic
2. Protocol-based - Exploit protocol weaknesses
3. Application-layer - Target specific features

Prevention:
✅ Use DDoS protection services (Cloudflare, AWS Shield)
✅ Rate limiting
✅ Load balancing
✅ Auto-scaling
✅ Monitoring and alerting

CHAPTER 3: SECURE CODING PRACTICES

A) INPUT VALIDATION:

CODE:

Rule: Never trust user input. Validate EVERYTHING!

```
// BAD - No validation
$email = $_POST['email'];  // Could be anything

// GOOD - Validate
$email = filter_var($_POST['email'], FILTER_VALIDATE_EMAIL);
if (!$email) {
    die("Invalid email");
}

// BAD
$age = $_POST['age'];  // Could be text

// GOOD
$age = filter_var($_POST['age'], FILTER_VALIDATE_INT);
if ($age === false || $age < 0) {
    die("Invalid age");
}
```

B) OUTPUT ESCAPING:

CODE:

Rule: Always escape output before displaying.

```
// BAD - XSS vulnerability
echo "<h1>Welcome, " . $_GET['name'] . "</h1>";

// GOOD - Escaped
echo "<h1>Welcome, " . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8') .
"</h1>";

// Python - BAD
print(f"Hello {user_input}")

// Python - GOOD
import html
print(f"Hello {html.escape(user_input)}")
```

C) PASSWORD SECURITY:

CODE:

Rule: Never store passwords in plain text.

```
// BAD - Plain text storage
$password = $_POST['password'];  // Stored as is

// GOOD - Hashing
$password = $_POST['password'];
$hashed = password_hash($password, PASSWORD_DEFAULT);
// Store $hashed in database

// Verify
if (password_verify($_POST['password'], $stored_hash)) {
    echo "Login successful";
}
```

// Strong Password Policy
✅ Minimum 12 characters
✅ Mix of uppercase, lowercase, numbers, symbols
✅ No common words or personal info
✅ Use password managers

D) SQL INJECTION PREVENTION:

CODE:

```
// BAD - Direct concatenation
$username = $_POST['username'];
$password = $_POST['password'];
$sql = "SELECT * FROM users WHERE username='$username' AND
password='$password'";

// GOOD - Prepared Statements (PHP PDO)
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username AND
password = :password");
$stmt->execute([':username' => $username, ':password' => $password]);

// GOOD - Python with SQLite
import sqlite3
conn = sqlite3.connect('db.sqlite')
cursor = conn.cursor()
cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?",
(username, password))
```

E) SESSION SECURITY:

CODE:

Secure Session Practices:
✅ Regenerate session ID after login
✅ Set session timeout (e.g., 30 minutes)
✅ Store only necessary data in session
✅ Use HTTPS for all session data
✅ Set secure and HttpOnly flags on cookies
✅ Use SameSite=Lax or Strict

Example (PHP):

```
session_start();
session_regenerate_id(true);  // After login

// Cookie settings
session_set_cookie_params([
    'lifetime' => 3600,
    'path' => '/',
    'domain' => 'example.com',
    'secure' => true,      // Only over HTTPS
    'httponly' => true,    // Not accessible by JS
    'samesite' => 'Lax'
]);
```

F) FILE UPLOAD SECURITY:

CODE:

File Upload Risks:
- Malicious files (PHP, EXE, viruses)
- Large files (DoS)
- Directory traversal

Secure File Upload:
✅ Validate file type (MIME type, extension)
✅ Validate file size
✅ Rename uploaded files (don't use user-supplied names)
✅ Store outside web root
✅ Scan for malware

```
if ($_FILES['file']['error'] === UPLOAD_ERR_OK) {
    $type = mime_content_type($_FILES['file']['tmp_name']);
```

```
    $allowed = ['image/jpeg', 'image/png', 'application/pdf'];

    if (!in_array($type, $allowed)) {
        die("File type not allowed");
    }

    if ($_FILES['file']['size'] > 5 * 1024 * 1024) {  // 5MB
        die("File too large");
    }

    $new_name = uniqid() . '.' . pathinfo($_FILES['file']['name'],
PATHINFO_EXTENSION);
    move_uploaded_file($_FILES['file']['tmp_name'], '/secure/path/' . $new_name);
}
```

G) HTTPS (SSL/TLS) IMPLEMENTATION:

CODE:

Why HTTPS?
✅ Encrypts all data between client and server
✅ Prevents eavesdropping and MITM attacks
✅ Builds user trust (shows padlock icon)

How to Implement:
1. Get SSL certificate (Let's Encrypt = free)
2. Configure web server (Apache/Nginx)
3. Redirect all HTTP to HTTPS
4. Enable HSTS (HTTP Strict Transport Security)

```
// Force HTTPS in .htaccess (Apache)
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

// Force HTTPS in PHP
if (!isset($_SERVER['HTTPS']) || $_SERVER['HTTPS'] !== 'on') {
    header('Location: https://' . $_SERVER['HTTP_HOST'] .
$_SERVER['REQUEST_URI']);
    exit();
}
```

CHAPTER 4: NETWORK SECURITY BASICS

==========

A) FIREWALLS:

CODE:

What is a Firewall?
- Network security system that monitors and controls traffic
- Based on predefined security rules

Types:
1. Network Firewall - Between networks
2. Host-based Firewall - On individual machines
3. Web Application Firewall (WAF) - For web apps

Common Rules:
✅ Block all traffic by default, allow specific ports
✅ Allow HTTP/HTTPS (ports 80, 443)
✅ Allow SSH (port 22) only from specific IPs
✅ Block known malicious IPs
✅ Block suspicious outbound connections

B) VPN (Virtual Private Network):

CODE:

What is VPN?
- Encrypted tunnel for internet traffic
- Masks IP address and location

Use Cases:
✅ Secure public Wi-Fi
✅ Access geo-restricted content
✅ Remote work
✅ Privacy and anonymity

C) IDS/IPS (Intrusion Detection/Prevention):

CODE:

IDS vs IPS:

- IDS: Detects and alerts (passive)
- IPS: Detects and blocks (active)

Common IDS/IPS:
- Snort (Open source)
- Suricata
- OSSEC (Host-based)

D) PORT SCANNING:

CODE:

Common Ports:

Port	Service
20, 21	FTP
22	SSH
23	Telnet (insecure)
25	SMTP
53	DNS
80	HTTP
110	POP3
143	IMAP
443	HTTPS
3306	MySQL
3389	RDP
5432	PostgreSQL
6379	Redis

Use Nmap to scan:
nmap -sS -p- target.com // SYN scan
nmap -sV target.com // Version detection

E) SECURITY HEADERS (Web Security):

CODE:

Important HTTP Security Headers:

```
// 1. Content-Security-Policy (CSP) - Prevents XSS
header("Content-Security-Policy: default-src 'self'");

// 2. X-Frame-Options - Prevents clickjacking
header("X-Frame-Options: DENY");
header("X-Frame-Options: SAMEORIGIN");

// 3. X-Content-Type-Options - Prevents MIME sniffing
header("X-Content-Type-Options: nosniff");

// 4. Strict-Transport-Security (HSTS) - Force HTTPS
header("Strict-Transport-Security: max-age=31536000; includeSubDomains");

// 5. Referrer-Policy - Controls referrer info
header("Referrer-Policy: strict-origin-when-cross-origin");

// 6. Permissions-Policy - Control browser features
header("Permissions-Policy: geolocation=(), microphone=(), camera=()");
```

==========

CHAPTER 5: ETHICAL HACKING TOOLS

==========

A) RECONNAISSANCE (Information Gathering):

Tools:

1. **Nmap** - Port scanning and network discovery
   ```
   nmap -A target.com   // Aggressive scan
   ```

2. **Shodan** - Search for internet-connected devices
   ```
   shodan search "port:22"
   ```

3. **WHOIS** - Domain registration info
   ```
   whois example.com
   ```

4. **TheHarvester** - Email/domain gathering
   ```
   theharvester -d example.com -b google
   ```

B) VULNERABILITY SCANNING:

Tools:

1. **Nessus** - Commercial vulnerability scanner
2. **OpenVAS** - Open-source alternative

3. Nikto - Web server scanner
4. WPScan - WordPress vulnerability scanner

C) WEB APPLICATION TESTING:

Tools:
1. Burp Suite - Web proxy + testing
2. OWASP ZAP - Open-source web scanner
3. SQLmap - SQL injection automation
sqlmap -u "http://target.com/page?id=1" --dbs

4. XSSer - XSS testing
5. Dirb - Directory brute force

D) PASSWORD CRACKING:

Tools:
1. John the Ripper - Password cracker
2. Hashcat - GPU-based password cracking
3. Hydra - Network authentication cracker
hydra -l user -P wordlist.txt target.com ssh

E) WIRELESS SECURITY:

Tools:
1. Aircrack-ng - Wi-Fi security testing
2. Kismet - Wireless network detector
3. Wireshark - Network packet analyzer

F) FRAMEWORKS:

1. Metasploit - Exploit framework
2. Empire - Post-exploitation
3. Cobalt Strike - Advanced threat simulation
4. Kali Linux - OS with 600+ security tools

═══
═══════

CHAPTER 6: SECURE COMMUNICATION

═══
═══════

A) ENCRYPTION BASICS:

CODE:
──
─────────

What is Encryption?
- Converting data into ciphertext using a key
- Decryption: Converting back to plaintext

Types:
1. Symmetric Encryption - Same key for encryption/decryption
- AES (Advanced Encryption Standard)
- DES (Data Encryption Standard) - Old
- Algorithms: AES-256, ChaCha20

2. Asymmetric Encryption - Different keys (Public/Private)
- RSA (Rivest-Shamir-Adleman)
- ECC (Elliptic Curve Cryptography)
- Used in SSL/TLS, digital signatures

B) SSL/TLS (HTTPS):

CODE:
_____

How SSL/TLS Works:
1. Client sends "Hello" with supported ciphers
2. Server responds with SSL certificate
3. Client verifies certificate (trusted CA)
4. Client generates session key
5. Session key encrypted with server's public key
6. Server decrypts with private key
7. Secure symmetric encryption begins

Check SSL/TLS:
- SSL Labs test: https://www.ssllabs.com/ssltest/
- Use strong ciphers (TLS 1.2 or 1.3)

C) ENCRYPTION IN CODE (Python Example):

CODE:
_____

```
# Symmetric Encryption (Fernet - Python)
from cryptography.fernet import Fernet

# Generate key
key = Fernet.generate_key()
cipher = Fernet(key)

# Encrypt
```

```
message = "This is secret!"
encrypted = cipher.encrypt(message.encode())

# Decrypt
decrypted = cipher.decrypt(encrypted).decode()
print(decrypted)  # This is secret!

# Password Hashing (Python)
import hashlib

password = "my_password"
hashed = hashlib.sha256(password.encode()).hexdigest()
print(hashed)  # 64 character hash
```

D) DIGITAL SIGNATURES:

CODE:

Purpose:
✅ Verify authenticity of a message
✅ Ensure message integrity
✅ Non-repudiation (sender can't deny)

How it works:
1. Sender hashes the message
2. Encrypts hash with private key (signature)
3. Receiver verifies with sender's public key

CHAPTER 7: SOCIAL ENGINEERING

What is Social Engineering?
- Psychological manipulation of people to get information
- People are the weakest link (95% of breaches involve human error)

A) TYPES OF SOCIAL ENGINEERING:

CODE:

---

1. Phishing - Fake emails/messages
2. Pretexting - Creating a false scenario

3. Baiting - Offering something attractive
4. Tailgating - Following someone into secure areas
5. Shoulder Surfing - Looking over someone's shoulder
6. Quid Pro Quo - Something in exchange for information
7. Dumpster Diving - Searching through trash

B) HOW TO PREVENT:

CODE:

✅ Security awareness training for all employees
✅ Verify identity before sharing information
✅ Don't share passwords with anyone
✅ Look for red flags (urgency, fear, authority)
✅ Report suspicious activities
✅ Implement multi-factor authentication (MFA)
✅ Regular security drills

==========

CHAPTER 8: INCIDENT RESPONSE

==========

A) 6 STEPS OF INCIDENT RESPONSE:

CODE:

---

1. Preparation - Develop policies and procedures
   ✅ Create incident response plan
   ✅ Setup monitoring and logging
   ✅ Security awareness training

2. Identification - Detect the incident
   ✅ Monitor logs and alerts
   ✅ User reports
   ✅ IDS/IPS alerts

3. Containment - Stop the spread
   ✅ Isolate affected systems
   ✅ Disconnect from network
   ✅ Kill processes

4. Eradication - Remove the threat

✅ Remove malware
✅ Patch vulnerabilities
✅ Delete malicious files

5. Recovery - Restore systems
✅ Restore from clean backups
✅ Rebuild systems if needed
✅ Monitor for recurrence

6. Lessons Learned - Improve
✅ Conduct post-mortem
✅ Update policies
✅ Implement new controls

B) BASIC INCIDENT RESPONSE COMMANDS:

CODE:

```
# Linux - Network investigation
netstat -tulpn    # Show listening ports
netstat -anp      # All connections
ps aux            # Running processes
last              # Login history
history           # Command history

# Check for rootkits
chkrootkit
rkhunter

# Check logs
tail -f /var/log/auth.log      # Login attempts
tail -f /var/log/syslog        # System logs
journalctl -xe                 # Systemd logs

# Windows - Network investigation
netstat -ano      # Active connections
tasklist          # Running processes
systeminfo        # System information
whoami            # Current user
wmic useraccount get name,sid  # List users
```

C) MALWARE ANALYSIS COMMANDS:

CODE:

```
# Linux
strings suspicious_file        # Extract readable strings
xxd suspicious_file | head     # Hex dump
file suspicious_file           # Detect file type
md5sum suspicious_file         # Calculate hash

# Check file permissions
ls -la file_name

# Monitor file changes
auditd - Watch system changes
tripwire - File integrity monitoring
```

CHAPTER 9: COMPLETE PROJECT - SECURE LOGIN SYSTEM

SECURE LOGIN SYSTEM (Full Implementation)

```
// config.php - Database and security settings
<?php
// Database connection
$pdo = new PDO("mysql:host=localhost;dbname=secure_app", "root", "");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// Session security
session_start();
session_regenerate_id(true);

// Security headers
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("Strict-Transport-Security: max-age=31536000");
header("Content-Security-Policy: default-src 'self'");

// Rate limiting (prevent brute force)
function checkRateLimit($ip) {
   global $pdo;
   $stmt = $pdo->prepare("SELECT COUNT(*) FROM login_attempts WHERE ip = :ip
AND attempt_time > NOW() - INTERVAL 15 MINUTE");
   $stmt->execute([':ip' => $ip]);
   $count = $stmt->fetchColumn();
```

```
    if ($count >= 5) {
        die("Too many login attempts. Please try again later.");
    }
}

function logAttempt($ip, $username) {
    global $pdo;
    $stmt = $pdo->prepare("INSERT INTO login_attempts (ip, username) VALUES (:ip,
:username)");
    $stmt->execute([':ip' => $ip, ':username' => $username]);
}
?>

// register.php - Secure registration
<?php
session_start();

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $username = trim($_POST['username'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $password = $_POST['password'] ?? '';
    $confirm = $_POST['confirm_password'] ?? '';

    // Validate
    $errors = [];

    if (empty($username)) $errors[] = "Username required";
    if (strlen($username) < 3) $errors[] = "Username min 3 characters";

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Invalid email";
    }

    if (strlen($password) < 8) $errors[] = "Password min 8 characters";
    if (!preg_match('/[A-Z]/', $password)) $errors[] = "Password needs uppercase";
    if (!preg_match('/[0-9]/', $password)) $errors[] = "Password needs number";
    if ($password !== $confirm) $errors[] = "Passwords don't match";

    if (empty($errors)) {
        $hashed = password_hash($password, PASSWORD_DEFAULT);
        $token = bin2hex(random_bytes(32));  // CSRF token

        try {
            $stmt = $pdo->prepare("INSERT INTO users (username, email, password,
csrf_token) VALUES (:username, :email, :password, :token)");
            $stmt->execute([
                ':username' => $username,
                ':email' => $email,
```

```
            ':password' => $hashed,
            ':token' => $token
        ]);
        $_SESSION['success'] = "Registration successful! Please login.";
        header("Location: login.php");
        exit();
    } catch (PDOException $e) {
        $errors[] = "Username or email already exists";
    }
  }
}
?>

// login.php - Secure login with MFA
<?php
session_start();

$ip = $_SERVER['REMOTE_ADDR'];
checkRateLimit($ip);

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
  $username = trim($_POST['username'] ?? '');
  $password = $_POST['password'] ?? '';

  if (empty($username) || empty($password)) {
    $error = "Username and password required";
  } else {
    $stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");
    $stmt->execute([':username' => $username]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($user && password_verify($password, $user['password'])) {
      // Login successful
      $_SESSION['user_id'] = $user['id'];
      $_SESSION['username'] = $user['username'];
      $_SESSION['ip'] = $ip;
      $_SESSION['user_agent'] = $_SERVER['HTTP_USER_AGENT'];
      session_regenerate_id(true);

      // Check for 2FA
      if ($user['two_factor_enabled']) {
        $_SESSION['needs_2fa'] = true;
        header("Location: 2fa.php");
      } else {
        header("Location: dashboard.php");
      }
      exit();
    } else {
```

```
            logAttempt($ip, $username);
            $error = "Invalid credentials";
        }
    }
}
?>

// 2fa.php - Two-Factor Authentication
<?php
session_start();

if (!isset($_SESSION['needs_2fa']) || $_SESSION['needs_2fa'] !== true) {
    header("Location: login.php");
    exit();
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $code = $_POST['code'] ?? '';

    // Verify TOTP (simplified - use library like robthree/twofactorauth)
    // $isValid = $totp->verifyCode($user['secret'], $code);

    if ($code == '123456') {  // Replace with real TOTP verification
        unset($_SESSION['needs_2fa']);
        $_SESSION['authenticated'] = true;
        header("Location: dashboard.php");
        exit();
    } else {
        $error = "Invalid 2FA code";
    }
}
?>

<!-- dashboard.php - Secure dashboard -->
<?php
session_start();

if (!isset($_SESSION['user_id']) || !isset($_SESSION['authenticated'])) {
    header("Location: login.php");
    exit();
}

// Verify session integrity
if ($_SESSION['ip'] !== $_SERVER['REMOTE_ADDR']) {
    session_destroy();
    header("Location: login.php");
    exit();
}
```

```
if ($_SESSION['user_agent'] !== $_SERVER['HTTP_USER_AGENT']) {
    session_destroy();
    header("Location: login.php");
    exit();
}

echo "Welcome, " . htmlspecialchars($_SESSION['username']) . "!";
?>

// Logout
<?php
session_start();
session_destroy();
header("Location: login.php");
exit();
?>

// SQL Tables
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    csrf_token VARCHAR(255) NOT NULL,
    two_factor_enabled BOOLEAN DEFAULT FALSE,
    two_factor_secret VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP,
    login_count INT DEFAULT 0
);

CREATE TABLE login_attempts (
    id INT PRIMARY KEY AUTO_INCREMENT,
    ip VARCHAR(45) NOT NULL,
    username VARCHAR(50),
    attempt_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE failed_logins (
    id INT PRIMARY KEY AUTO_INCREMENT,
    ip VARCHAR(45) NOT NULL,
    attempt_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

CHAPTER 10: QUICK REFERENCE - CYBER SECURITY

COMMON PORTS (Know Them):
20,21 - FTP | 22 - SSH | 23 - Telnet | 25 - SMTP
53 - DNS | 80 - HTTP | 110 - POP3 | 143 - IMAP
443 - HTTPS | 3306 - MySQL | 3389 - RDP | 5432 - PostgreSQL

SECURITY HEADERS:
Content-Security-Policy, X-Frame-Options, X-Content-Type-Options
Strict-Transport-Security, Referrer-Policy, Permissions-Policy

ENCRYPTION COMMANDS:
Linux: openssl enc -aes-256-cbc -salt -in file.txt -out file.enc
Windows: cipher /e (encrypt folder)
GPG: gpg -c file.txt (encrypt), gpg file.txt.gpg (decrypt)

SECURE CODING RULES:
✅ Validate all input
✅ Escape all output
✅ Use prepared statements for SQL
✅ Hash passwords (bcrypt/PBKDF2/Argon2)
✅ Use HTTPS everywhere
✅ Implement MFA
✅ Regular security updates
✅ Least privilege principle

INCIDENT RESPONSE COMMANDS:
Linux: netstat -tulpn, ps aux, last, history, chkrootkit
Windows: netstat -ano, tasklist, systeminfo, whoami

CERTIFICATE OF COMPLETION

This certifies that the reader has completed

CYBER SECURITY BASICS

By
KRUTIK 🚨 CYBER EXPERT

Topics Covered:
✓ Cyber Security Introduction (CIA Triad, Hacker Types)
✓ Common Cyber Attacks (Phishing, Malware, MITM, SQL Injection, XSS, CSRF, DDoS)

✓ **Secure Coding Practices (Input Validation, Output Escaping, Password Security)**
✓ **SQL Injection Prevention (Prepared Statements)**
✓ **Session Security (Secure Cookies, MFA)**
✓ **File Upload Security**
✓ **HTTPS Implementation (SSL/TLS)**
✓ **Network Security (Firewalls, VPN, IDS/IPS, Port Scanning)**
✓ **Security Headers (CSP, HSTS, X-Frame-Options)**
✓ **Ethical Hacking Tools (Nmap, Burp Suite, Metasploit)**
✓ **Secure Communication (Encryption, Digital Signatures)**
✓ **Social Engineering (Types and Prevention)**
✓ **Incident Response (6 Steps)**
✓ **Complete Secure Login System (Registration, Login, MFA, Rate Limiting)**
✓ **Quick Reference (Ports, Headers, Commands)**

"Hack the Planet. Securely."

NOTE: This PDF is completely FREE. If you paid for it, you got scammed.
Share it with everyone who wants to learn Cyber Security! 🚀
